

LAPORAN TUGAS

**WEB SERVER
(TIF 330805)**

MINGGU KE 8



ACARA 15

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM: E41250631

GOL: A

**PROGRAM STUDI DI LUAR KAMPUS UTAMA (PSDKU) POLITEKNIK NEGERI
JEMBER
TEKNIK INFORMATIKA KAB. NGANJUK JL. JENDRAL GATOT SUBROTO NO
106, KAUMAN, KECAMATAN NGANJUK, KABUPATEN NGANJUK
JURUSAN TEKNOLOGI INFORMASI POLITEKNIK NEGERI JEMBER TAHUN
2026**

PENDAHULUAN

Pada praktikum sebelumnya, data mahasiswa hanya dapat diakses melalui halaman web yang dirender oleh server. Aplikasi lain, misalnya aplikasi mobile atau sistem pihak ketiga, tidak dapat memakai data tersebut dengan mudah karena yang dikembalikan berupa HTML. Untuk kebutuhan ini, server menyediakan **API (Application Programming Interface)** yang mengembalikan data dalam format yang mudah dibaca program, yaitu **JSON**.

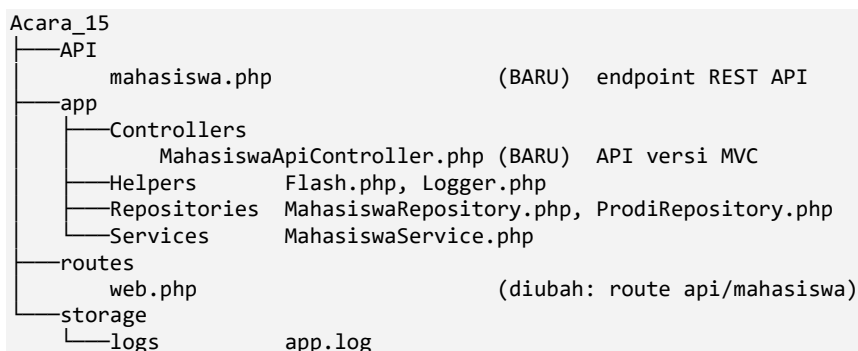
Gaya perancangan API yang umum dipakai adalah **REST**. Pada REST, setiap sumber data diakses melalui sebuah URL (endpoint), dan jenis operasinya ditentukan oleh **HTTP method**: GET untuk membaca data, POST untuk menambah data, serta PUT dan DELETE untuk mengubah dan menghapus data. Hasil permintaan dinyatakan melalui **kode status HTTP**, misalnya 200 (OK), 201 (Created), 400 (Bad Request), 404 (Not Found), 405 (Method Not Allowed), 422 (Unprocessable Entity), dan 500 (Internal Server Error).

Praktikum ini membuat API data mahasiswa yang mendukung pembacaan seluruh data, pembacaan satu data berdasarkan id, dan penambahan data melalui body JSON. Keamanan dan penanganan error dari praktikum sebelumnya tetap diterapkan: input divalidasi, query memakai prepared statement, dan error server dicatat ke log tanpa membocorkan detail teknis kepada pengguna. Pengujian dilakukan menggunakan **Postman**, yaitu aplikasi untuk mengirim request HTTP dan melihat responsnya.

IMPLEMENTASI

1. Menyiapkan folder Acara_15

Project dari Acara 14 digunakan sebagai dasar pengerjaan pada folder Acara_15. Ditambahkan folder API berisi endpoint mahasiswa.php dan file MahasiswaApiController.php pada folder Controllers.



[TEMPEL SCREENSHOT: struktur folder Acara_15 di Explorer VS Code]

2. Membuat endpoint API/mahasiswa.php

File API/mahasiswa.php dibuat sebagai endpoint REST. Header Content-Type: application/json dikirim agar klien membaca respons sebagai JSON. Fungsi respond() menjadi satu-satunya tempat keluaran: ia mengatur kode status HTTP lalu mencetak JSON dengan format seragam success, message, dan data. Fungsi rapikan() mengubah id, prodi_id, dan angkatan menjadi bilangan bulat agar tipe data JSON benar.

Alur endpoint ditentukan oleh HTTP method:

Method	Permintaan	Perilaku
GET	/API/mahasiswa.php	Mengembalikan seluruh data mahasiswa (200)
GET	/API/mahasiswa.php?id=6	Mengembalikan satu mahasiswa (200), id tidak valid (400), tidak ditemukan (404)
POST	/API/mahasiswa.php + body JSON	Menambah mahasiswa (201), JSON tidak valid (400), data tidak valid (422)
Lainnya	-	Ditolak dengan kode 405 (Method tidak diizinkan)
Error server	-	Dicatat ke app.log, pengguna menerima pesan umum (500)

```

<?php
header('Content-Type: application/json; charset=utf-8');

require_once __DIR__ . '/../app/Core/Database.php';
require_once __DIR__ . '/../app/Helpers/Logger.php';

function respond(int $status, bool $success, string $message, $data = null): void
{
    http_response_code($status);
}

```

```

    echo json_encode(
        ['success' => $success, 'message' => $message, 'data' => $data],
        JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
    );
    exit;
}

function rapikan(array $row): array
{
    $row['id'] = (int) $row['id'];
    $row['prodi_id'] = (int) $row['prodi_id'];
    $row['angkatan'] = (int) $row['angkatan'];
    return $row;
}

try {
    $pdo = Database::getInstance()->getConnection();
    $method = $_SERVER['REQUEST_METHOD'];

    // ===== GET: daftar mahasiswa atau satu mahasiswa (?id=1) =====
    if ($method === 'GET') {
        if (isset($_GET['id'])) {
            $id = (int) $_GET['id'];
            if ($id <= 0) {
                respond(400, false, 'Parameter id tidak valid');
            }

            $stmt = $pdo->prepare(
                'SELECT id, nim, nama, email, prodi_id, angkatan, status
                FROM mahasiswa WHERE id = :id'
            );
            $stmt->execute(['id' => $id]);
            $row = $stmt->fetch(PDO::FETCH_ASSOC);

            if (!$row) {
                respond(404, false, 'Data mahasiswa tidak ditemukan');
            }
            respond(200, true, 'Data berhasil diambil', rapikan($row));
        }

        $stmt = $pdo->query(
            'SELECT id, nim, nama, email, prodi_id, angkatan, status
            FROM mahasiswa ORDER BY id ASC'
        );
        $data = array_map('rapikan', $stmt->fetchAll(PDO::FETCH_ASSOC));
        respond(200, true, 'Data berhasil diambil', $data);
    }

    // ===== POST: tambah mahasiswa (body JSON) =====
    if ($method === 'POST') {
        $input = json_decode(file_get_contents('php://input'), true);
        if (json_last_error() !== JSON_ERROR_NONE || !is_array($input)) {
            respond(400, false, 'Body harus berupa JSON yang valid');
        }

        $nim = trim($input['nim'] ?? '');
        $nama = trim($input['nama'] ?? '');
        $email = trim($input['email'] ?? '');
        $prodiId = (int) ($input['prodi_id'] ?? 1);
        $angkatan = (int) ($input['angkatan'] ?? date('Y'));
        $status = $input['status'] ?? 'aktif';

        $errors = [];
        if ($nim === '') {
            $errors['nim'] = 'NIM wajib diisi';
        } elseif (!ctype_digit($nim)) {

```

```

        $errors['nim'] = 'NIM harus berupa angka';
    } else {
        $cek = $pdo->prepare('SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim');
        $cek->execute(['nim' => $nim]);
        if ((int) $cek->fetchColumn() > 0) {
            $errors['nim'] = 'NIM sudah terdaftar';
        }
    }
    if ($nama === '') {
        $errors['nama'] = 'Nama wajib diisi';
    }
    if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors['email'] = 'Format email tidak valid';
    }
    $cekProdi = $pdo->prepare('SELECT COUNT(*) FROM prodi WHERE id = :id');
    $cekProdi->execute(['id' => $prodiId]);
    if ($prodiId <= 0 || (int) $cekProdi->fetchColumn() === 0) {
        $errors['prodi_id'] = 'Program studi tidak tersedia';
    }
    if ($angkatan < 2000 || $angkatan > (int) date('Y')) {
        $errors['angkatan'] = 'Angkatan tidak valid';
    }
    if (!in_array($status, ['aktif', 'cuti', 'lulus'], true)) {
        $errors['status'] = 'Status tidak valid';
    }

    if (!empty($errors)) {
        respond(422, false, 'Data tidak valid', $errors);
    }

    $stmt = $pdo->prepare(
        'INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan, status)
        VALUES (:nim, :nama, :email, :prodi_id, :angkatan, :status)'
    );
    $stmt->execute([
        'nim' => $nim,
        'nama' => $nama,
        'email' => $email,
        'prodi_id' => $prodiId,
        'angkatan' => $angkatan,
        'status' => $status,
    ]);

    respond(201, true, 'Data mahasiswa berhasil ditambahkan', [
        'id' => (int) $pdo->lastInsertId(),
        'nim' => $nim,
        'nama' => $nama,
        'email' => $email,
    ]);
}

respond(405, false, 'Method tidak diizinkan');
} catch (Throwable $e) {
    Logger::error('api/mahasiswa.php', $e);
    respond(500, false, 'Terjadi kesalahan pada server');
}

```

Pada method POST, body dibaca dari `php://input` dan didekode dengan `json_decode()`. Jika bukan JSON yang valid, server membalas 400. Data yang tidak lolos validasi (NIM wajib dan harus angka, NIM tidak boleh sama, email valid, prodi tersedia, angkatan dan status valid) dibalas 422 beserta daftar error per kolom. Seluruh query memakai prepared statement, dan blok catch mencatat error ke Logger serta hanya mengirim pesan "Terjadi kesalahan pada server" kepada klien.

3. Format respons JSON

Seluruh respons memakai struktur yang sama sehingga mudah diproses oleh klien. Contoh respons saat penambahan data berhasil:

```
{
  "success": true,
  "message": "Data mahasiswa berhasil ditambahkan",
  "data": {
    "id": 6,
    "nim": "23004",
    "nama": "Dewi",
    "email": "dewi@gmail.com"
  }
}
```

4. Membuat versi terintegrasi MVC (MahasiswaApiController)

Selain endpoint mandiri, dibuat juga MahasiswaApiController yang memakai MahasiswaRepository dan MahasiswaService dari praktikum sebelumnya. Dengan cara ini, validasi pada API sama dengan validasi pada halaman web karena sama-sama memanggil Service. Controller ini memiliki method index() untuk GET dan store() untuk POST.

```
<?php
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';
require_once __DIR__ . '/../Services/MahasiswaService.php';
require_once __DIR__ . '/../Entities/Mahasiswa.php';
require_once __DIR__ . '/../Helpers/Logger.php';

class MahasiswaApiController
{
    private MahasiswaRepository $repo;
    private MahasiswaService $service;

    public function __construct(MahasiswaRepository $repo, MahasiswaService $service)
    {
        $this->repo = $repo;
        $this->service = $service;
    }

    private function json(int $status, bool $success, string $message, $data = null): void
    {
        http_response_code($status);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode(
            ['success' => $success, 'message' => $message, 'data' => $data],
            JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
        );
    }

    private function format($row): array
    {
        $prodiNama = null;
        $mahasiswa = $row;

        // all() mengembalikan ['mahasiswa' => Mahasiswa, 'prodi_nama' => '...']
        if (is_array($row) && isset($row['mahasiswa'])) {
            $mahasiswa = $row['mahasiswa'];
            $prodiNama = $row['prodi_nama'] ?? null;
        }

        return [
            'id' => $mahasiswa->getId(),
```

```

        'nim' => $mahasiswa->getNim(),
        'nama' => $mahasiswa->getNama(),
        'email' => $mahasiswa->getEmail(),
        'prodi_id' => $mahasiswa->getProdiId(),
        'prodi_nama' => $prodiNama,
        'angkatan' => $mahasiswa->getAngkatan(),
        'status' => $mahasiswa->getStatus(),
    ];
}

// GET /api/mahasiswa atau GET /api/mahasiswa?id=1
public function index(): void
{
    try {
        if (isset($_GET['id'])) {
            $this->show((int) $_GET['id']);
            return;
        }

        $keyword = trim($_GET['q'] ?? '');
        $rows = $this->repo->all($keyword !== '' ? $keyword : null);
        $data = array_map([$this, 'format'], $rows);

        $this->json(200, true, 'Data berhasil diambil', $data);
    } catch (Throwable $e) {
        Logger::error('MahasiswaApiController::index', $e);
        $this->json(500, false, 'Terjadi kesalahan pada server');
    }
}

private function show(int $id): void
{
    if ($id <= 0) {
        $this->json(400, false, 'Parameter id tidak valid');
        return;
    }

    $mahasiswa = $this->repo->find($id);
    if ($mahasiswa === null) {
        $this->json(404, false, 'Data mahasiswa tidak ditemukan');
        return;
    }

    $this->json(200, true, 'Data berhasil diambil', $this->format($mahasiswa));
}

// POST /api/mahasiswa (body JSON)
public function store(): void
{
    try {
        $input = json_decode(file_get_contents('php://input'), true);

        if (json_last_error() !== JSON_ERROR_NONE || !is_array($input)) {
            $this->json(400, false, 'Body harus berupa JSON yang valid');
            return;
        }

        $result = $this->service->create($input);

        if ($result['success']) {
            $this->json(201, true, 'Data mahasiswa berhasil ditambahkan');
            return;
        }

        $this->json(422, false, 'Data tidak valid', $result['errors']);
    }
}

```

```

        } catch (Throwable $e) {
            Logger::error('MahasiswaApiController::store', $e);
            $this->json(500, false, 'Terjadi kesalahan pada server');
        }
    }
}

```

Route dan perangkaian dependency pada routes/web.php ditambahkan sebagai berikut, sehingga API juga dapat diakses melalui URL /api/mahasiswa.

```

['GET', 'api/mahasiswa', 'MahasiswaApiController', 'index', false],
['POST', 'api/mahasiswa', 'MahasiswaApiController', 'store', false],

} elseif ($controllerName === 'MahasiswaApiController') {
    try {
        $database = Database::getInstance();
        $repository = new MahasiswaRepository($database);
        $prodiRepository = new ProdiRepository($database);
        $service = new MahasiswaService($repository, $prodiRepository);
        $controller = new MahasiswaApiController($repository, $service);
    } catch (Throwable $e) {
        Logger::error('web.php::MahasiswaApiController', $e);
        http_response_code(500);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode([
            'success' => false,
            'message' => 'Terjadi kesalahan pada server',
            'data' => null,
        ]);
        return;
    }
} else {

```

5. Menguji API menggunakan Postman

Pengujian dilakukan dengan Postman terhadap endpoint `http://localhost/tugas%20acara_cya/Acara_15/api/mahasiswa.php`. Body request JSON dikirim melalui tab Body dengan pilihan raw. Skenario dan hasilnya dijelaskan pada bagian Hasil Pengujian.

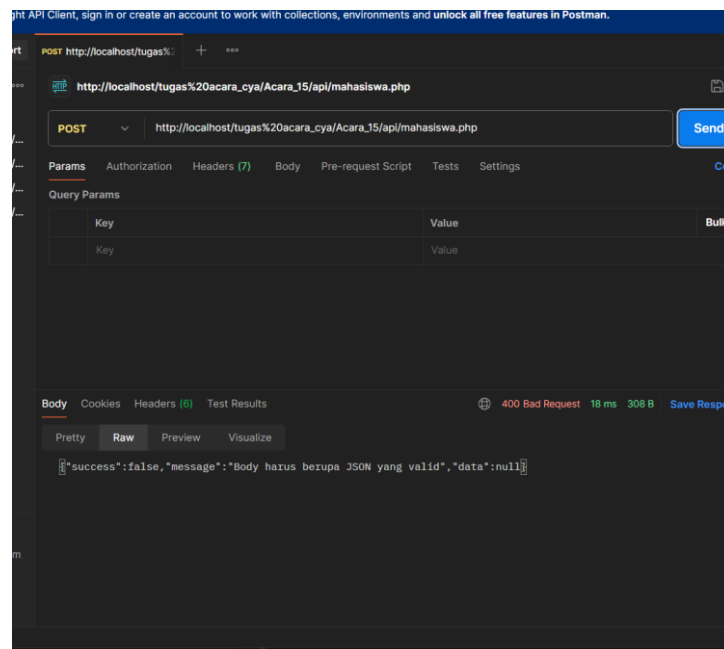
HASIL PENGUJIAN

Ringkasan pengujian endpoint ditampilkan pada tabel berikut.

No	Skenario	Hasil yang Diharapkan	Status HTTP	Hasil
1	POST tanpa body JSON	Pesan "Body harus berupa JSON yang valid"	400	Sesuai
2	POST data valid (NIM 23004, Dewi)	Data tersimpan, id baru dikembalikan	201	Sesuai
3	GET seluruh data	Daftar mahasiswa dalam array JSON	200	Sesuai
4	GET satu data (?id=6)	Data mahasiswa dengan id 6	200	Sesuai
5	POST dengan NIM yang sudah terdaftar	Pesan "NIM sudah terdaftar"	422	Perlu screenshot
6	GET dengan id yang tidak ada	Pesan "Data mahasiswa tidak ditemukan"	404	Perlu screenshot

a. POST tanpa body JSON (400 Bad Request)

Request POST dikirim tanpa body JSON. Server menolak dengan kode 400 dan pesan "Body harus berupa JSON yang valid". Ini membuktikan server tidak memproses permintaan yang formatnya salah.

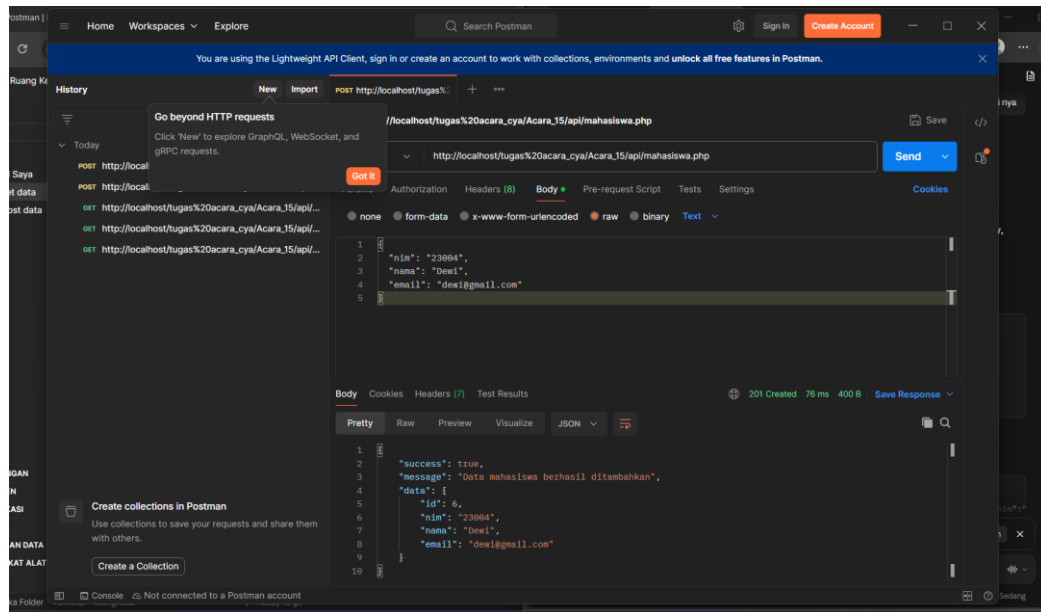


Gambar 1. Respons 400 Bad Request pada POST tanpa body JSON

b. POST data valid (201 Created)

Body JSON berisi nim "23004", nama "Dewi", dan email "dewi@gmail.com" dikirim melalui POST. Server membalas 201 Created dengan id baru (6). Karena prodi_id, angkatan, dan

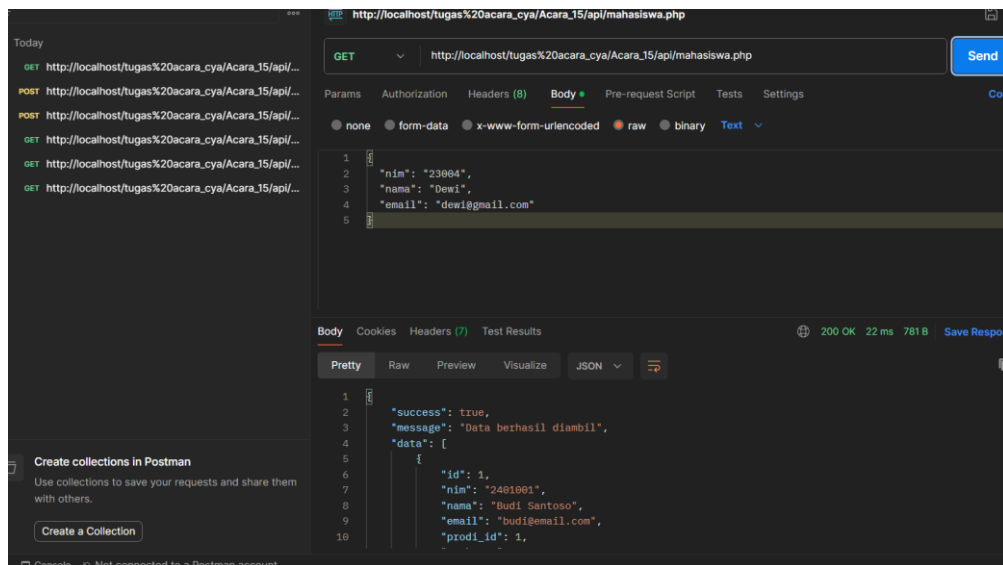
status tidak dikirim, server memakai nilai bawaan yaitu prodi 1, angkatan tahun berjalan, dan status "aktif".



Gambar 2. Respons 201 Created pada POST data valid

c. GET seluruh data (200 OK)

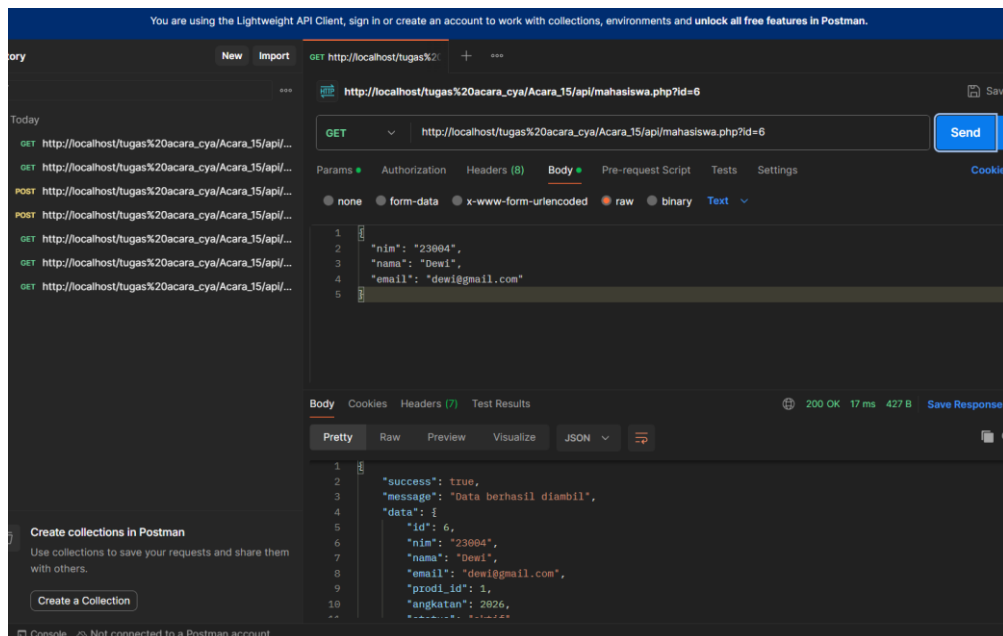
Request GET tanpa parameter mengembalikan seluruh data mahasiswa dalam bentuk array JSON, termasuk data Dewi yang baru ditambahkan.



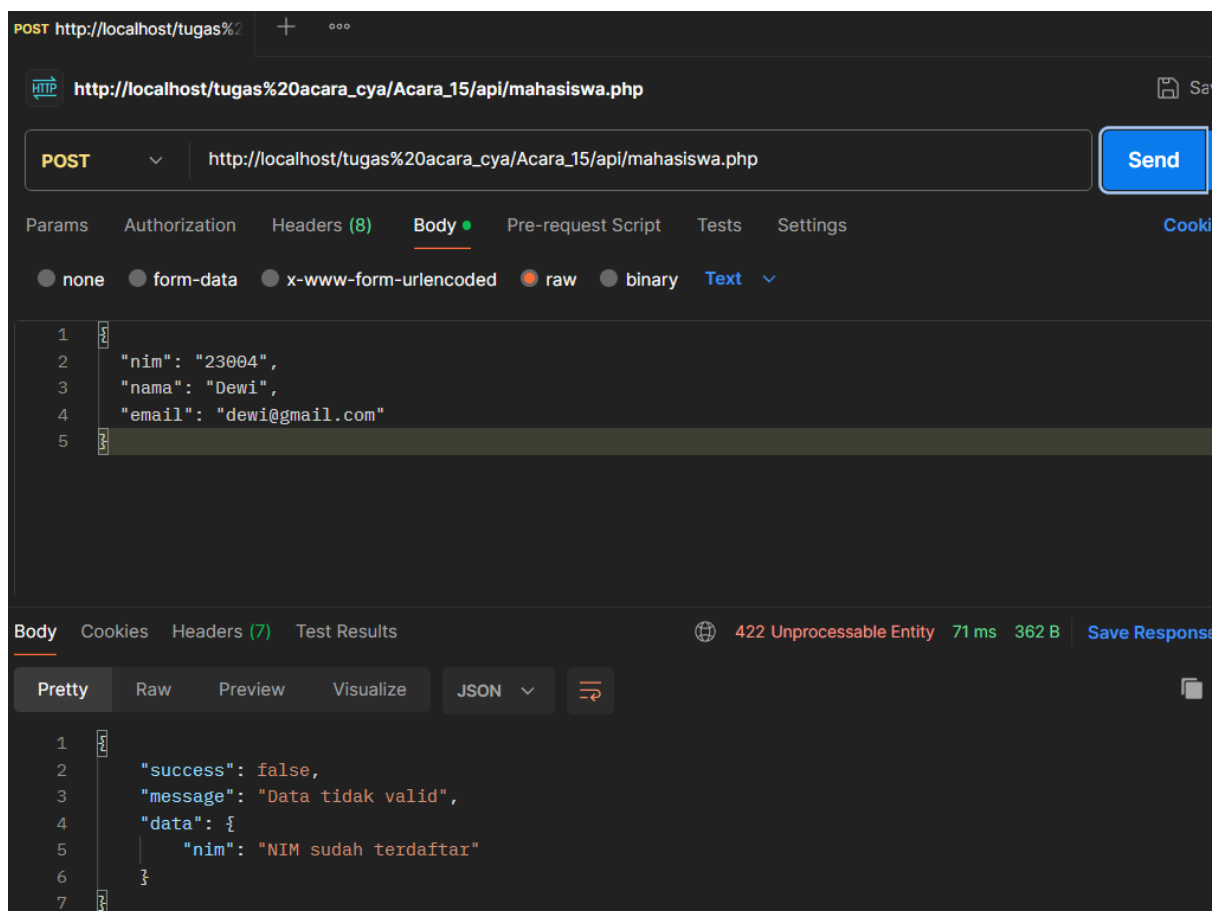
Gambar 3. Respons 200 OK pada GET seluruh data

d. GET satu data berdasarkan id (200 OK)

Request GET dengan parameter `?id=6` mengembalikan satu data, yaitu Dewi dengan angkatan 2026. Hal ini membuktikan data hasil POST benar-benar tersimpan di database.

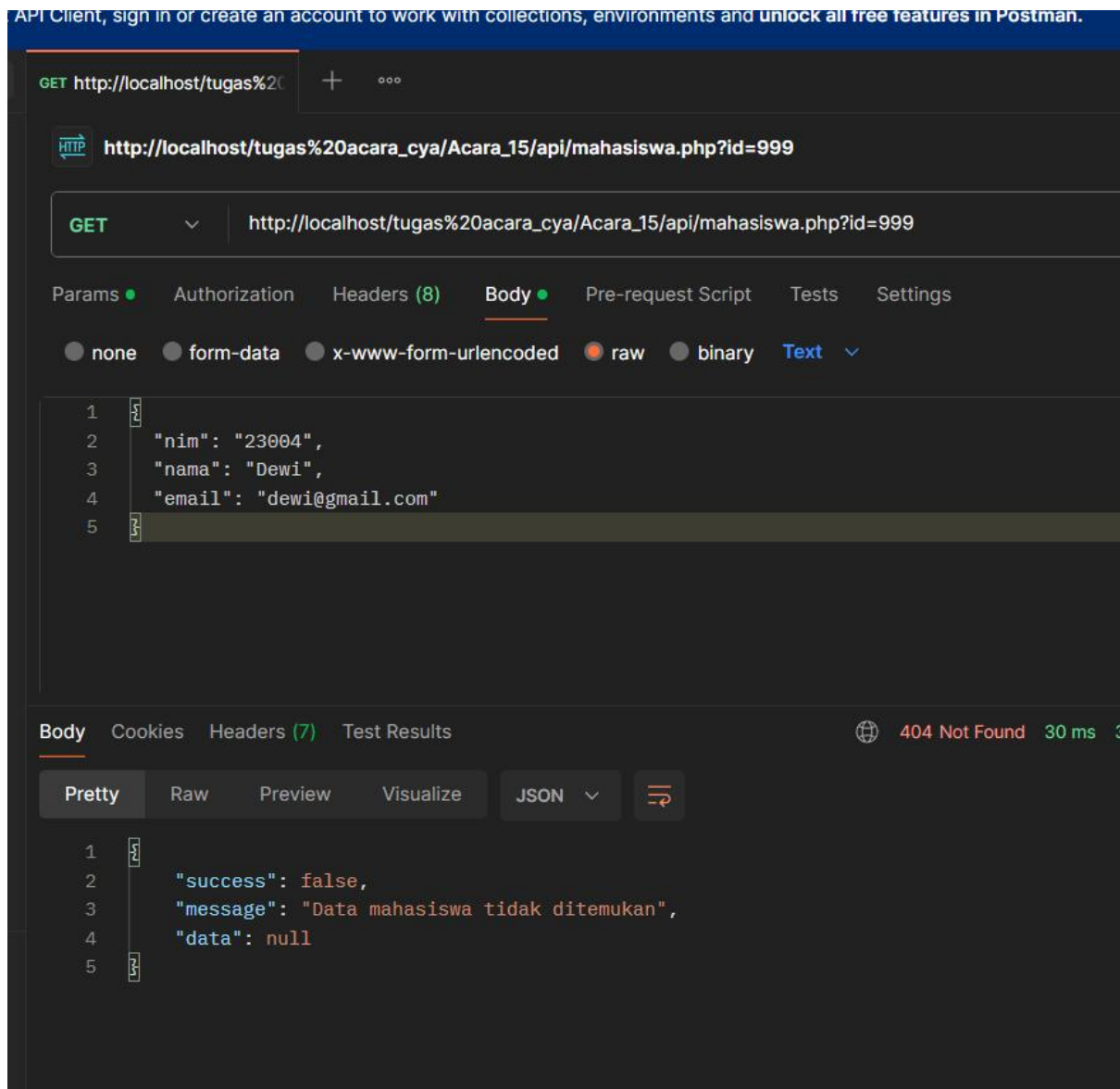


Gambar 4. Respons 200 OK pada GET dengan `?id=6`

e. POST dengan NIM yang sudah terdaftar (422)

Kirim ulang body JSON yang sama (NIM 23004) lewat POST. Server seharusnya membalas 422 dengan pesan error pada kolom nim.

f. GET dengan id yang tidak ada (404)



Kirim GET dengan ?id=999. Server seharusnya membalas 404 dengan pesan "Data mahasiswa tidak ditemukan".

ANALISIS

Endpoint API mengubah cara data mahasiswa diakses. Alih-alih mengembalikan halaman HTML, server mengembalikan JSON yang dapat dibaca aplikasi lain. Hasil pengujian menunjukkan GET mengembalikan data dengan kode 200, sedangkan POST yang berhasil mengembalikan 201 Created beserta id data baru. Pemakaian kode status yang tepat membuat klien dapat mengetahui hasil permintaan tanpa harus membaca isi pesannya.

Format respons yang seragam (`success`, `message`, `data`) dihasilkan oleh satu fungsi `respond()`. Dengan cara ini setiap kondisi, baik sukses maupun error, memiliki bentuk yang sama sehingga klien cukup menulis satu cara membaca respons.

Pengujian POST tanpa body menghasilkan 400 Bad Request, yang menunjukkan validasi dilakukan sebelum data diproses. Untuk data yang formatnya benar tetapi isinya tidak valid, API membalas 422 beserta daftar error per kolom. Perbedaan 400 dan 422 ini penting: 400 berarti permintaan tidak dapat dibaca, sedangkan 422 berarti permintaan dapat dibaca tetapi datanya tidak memenuhi aturan.

Pada pengujian POST, kolom `prodi_id`, `angkatan`, dan `status` tidak dikirim, tetapi data tetap tersimpan karena server memakai nilai bawaan (`prodi 1`, `angkatan tahun berjalan`, `status "aktif"`). Hasil GET dengan `?id=6` memperlihatkan angkatan 2026, sesuai nilai bawaan tersebut.

Aspek keamanan dari Acara 14 tetap berjalan. Query memakai prepared statement sehingga terlindung dari SQL Injection, dan setiap exception dicatat melalui Logger ke `app.log`, sedangkan klien hanya menerima pesan umum dengan kode 500 tanpa detail teknis database.

Dibuatnya `MahasiswaApiController` yang memanggil `MahasiswaService` menunjukkan bahwa aturan validasi dapat dipakai bersama oleh halaman web dan API. Aturan bisnis cukup ditulis satu kali di Service, sehingga perilaku web dan API tetap konsisten.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. REST API memungkinkan data mahasiswa diakses oleh aplikasi lain melalui URL dan HTTP method, dengan data dalam format JSON.
2. Method GET digunakan untuk membaca data (seluruh data atau satu data dengan ?id=), dan method POST digunakan untuk menambah data dengan body JSON.
3. Kode status HTTP yang tepat (200, 201, 400, 404, 405, 422, dan 500) menyampaikan hasil permintaan dengan jelas kepada klien.
4. Format respons yang seragam (success, message, data) memudahkan klien memproses hasil.
5. Validasi input, prepared statement, dan logging dari Acara 14 tetap diterapkan pada API, sehingga detail error teknis tidak dikirim ke pengguna.
6. Postman dapat dipakai untuk menguji endpoint API, mulai dari request, body JSON, hingga memeriksa kode status dan isi respons.

DAFTAR PUSTAKA

1. Politeknik Negeri Jember. 2026. Modul Praktikum Workshop SI Web Server (TIF330805), Acara 15.
2. The PHP Group. PHP Manual: json_encode. <https://www.php.net/manual/en/function.json-encode.php>
3. MDN Web Docs. HTTP response status codes. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
4. Postman. Postman Learning Center. <https://learning.postman.com>